

Lecture 8

Type Variance and Function Subtyping

CS 12 Team 25.2

A.Y. 2025–2026, 2nd Sem

Department of Computer Science

College of Engineering

University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

Analogy

How are these related in terms of *subtypes*?

Type	Values
Drink	  
Liquid	  

Analogy

Drink <: Liquid?

-  as liquid?
-  as liquid?
-  as liquid?

Liquid <: Drink?

-  as drink?
-  as drink?
-  as drink?

Analogy

Drink <: Liquid!

Type	Values
Drink	  
Liquid	     

Why are we talking about drinks and liquids? We'll see later!

Generic Types

Generic type: type parameterized by other types

- `list[T]`; T is the *type parameter*
 - `list[int]`
 - `list[str]`
 - `list[list[int]]`
 - ...

Analogy

How are *these* related in terms of subtypes?

Type	Values
List[Drink]	[    ], [  ]
List[Liquid]	[ ], [   ]

Analogy

List[Drink] <: List[Liquid]?

- [

List[Liquid] <: List[Drink]?

- [

Analogy

List[Drink] <: List[Liquid]?

- [Not always!

List[Liquid] <: List[Drink]?

- [It depends!

Analogy

List[Drink] <: List[Liquid]?

- [Not always!

List[Liquid] <: List[Drink]?

- [

Analogy

List[Drink] can *sometimes* be used as List[Liquid]...

```
def print_volumes(liquids: List[Liquid]):  
    for liquid in liquids:  
        print(liquid.volume)
```

```
list_of_drinks = [
```

print_volumes(list_of_drinks) should be allowed if we allow
List[Drink] <: List[Liquid] (correct)

```
print_volumes(list_of_drinks) # Valid; all drinks have volume
```

Analogy

...but *sometimes* is not good enough!

```
def add_gasoline(liquids: List[Liquid]):  
    liquids.append()
```

```
list_of_drinks = [   
```

Is allowing `add_gasoline(list_of_drinks)` a problem?

```
# Allowed if `List[Drink] <: List[Liquid]`  
add_gasoline(list_of_drinks)  
print(list_of_drinks) # [    
```

Analogy

Allowing `List[Drink] <: List[Liquid]` is *potentially fatal*!

```
def serve_drinks(drinks: List[Drink]):  
    for drink in drinks:  
        customer.drink(drink)    # In scope elsewhere
```

```
list_of_drinks = [)`
# is executed!
```

Analogy

`List[Drink] <: List[Liquid]` is not valid since you can *modify* Lists

- Any subtype of `Liquid` can be added to `List[Drink]`
 - 🚰 can be added to `List[Drink]`
 - Breaks types since `Liquid </: Drink` (other way around!)
- What if `List` cannot be written to?
 - `ReadOnlyList`: Immutable version of `List`
- Is it possible to break type system guarantees if we allow `ReadOnlyList[Drink] <: ReadOnlyList[Liquid]`?

Analogy

`ReadOnlyList[Drink]` can *always* be used as
`ReadOnlyList[Liquid]`

```
def print_volumes(ls: ReadOnlyList[Liquid]):  
    for liquid in ls:  
        print(liquid.volume)
```

```
# ReadOnlyList[Drink]  
readonly_drinks = [   
```

Analogy

```
# Cannot define this!
def add_gasoline(ls: ReadOnlyList[Liquid]):
    ls.append() # Error: `ReadOnlyList`
                                # has no `.append` method!
```

- Write operations are disallowed!
- `ReadOnlyList[Drink] <: ReadOnlyList[Liquid]` is valid only if:
 - `print_volumes(readonly_drinks)` (read) is allowed
 - `add_gasoline(readonly_drinks)` (write) is disallowed

Analogy

Subtyping relations so far:

- `Drink <: Liquid`
- `List[Drink] </: List[Liquid]`
- `ReadOnlyList[Drink] <: ReadOnlyList[Liquid]`

Analogy

List[Drink] <: List[Liquid]?

- [    ] as list of liquids? Not always!

List[Liquid] <: List[Drink]?

- [   ] as list of drinks? **It depends!**

Analogy

Of course, `List[Liquid] <: List[Drink]` should not be allowed...

```
def serve_drinks(drinks: List[Drink]):  
    for drink in drinks:  
        customer.drink(drink)
```

```
list_of_liquids = [   
```

`serve_drinks(list_of_liquids)` should not be allowed, so
`List[Liquid] </: List[Drink]`

Analogy

...but there are still cases where it could be!

```
# Both `Drink` and `Liquid` have `.volume`  
def add_beer(drinks: List[Drink]):  
    drinks.append()
```

```
list_of_liquids = [    
add_beer(list_of_liquids)  
print(list_of_liquids) # [   # still consistent with `List[Liquid]`!
```

Analogy

`List[Liquid] <: List[Drink]` is not valid if can *read* from `List`

- Not all elements in `List[Liquid]` may have `Drink`-specific methods like `.drink()`

`List[Liquid] <: List[Drink]` is valid when `List` is *write-only*

- `Drink` can be added to `List[Drink]` and `List[Liquid]`

`WriteOnlyList`: `List` that cannot be read from, but can be written to

- `WriteOnlyList[Liquid] <: WriteOnlyList[Drink]` is valid

Analogy

Subtyping relations formed:

- `Drink <: Liquid`
- `List[Drink] </: List[Liquid]`
- `ReadOnlyList[Drink] <: ReadOnlyList[Liquid]`
- `List[Liquid] </: List[Drink]`
- `WriteOnlyList[Liquid] <: WriteOnlyList[Drink]`

Subtyping Patterns

What happened to the original subtyping relation `Drink <: Liquid` after introducing `ReadOnlyList`?



Covariance

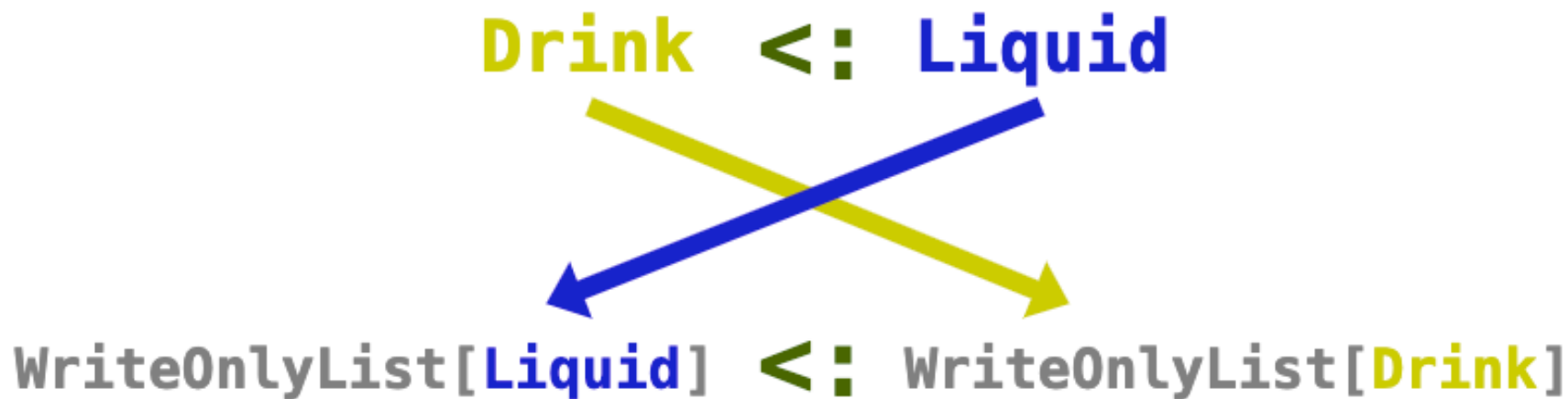
Covariance: Original subtyping direction is *followed* in subtyping between “wrapped” versions (co-)



`ReadOnlyList[T]` is *covariant* on `T`

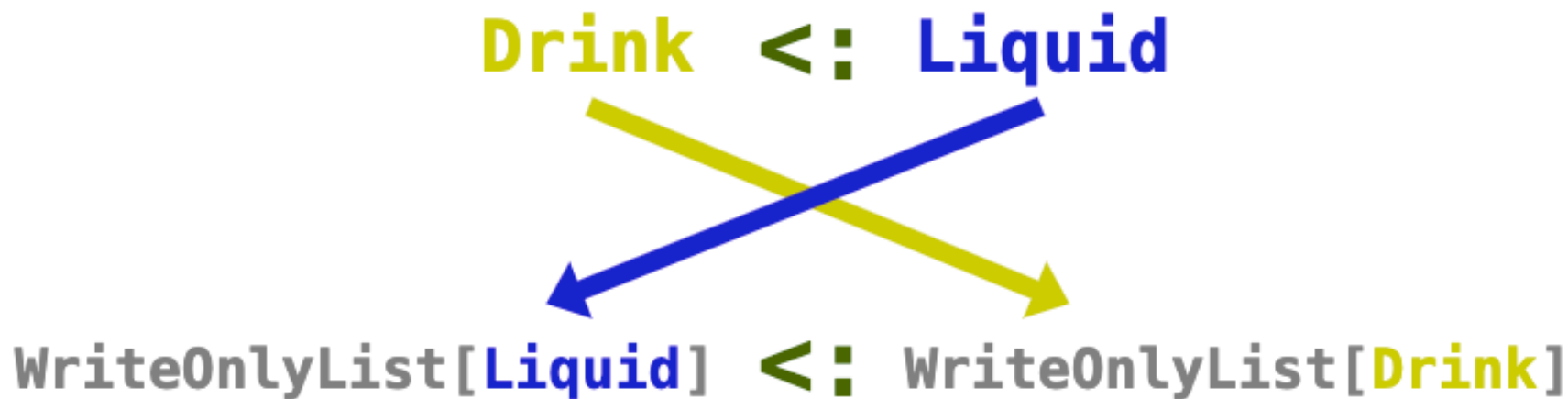
Subtyping Patterns

What happened to the original subtyping relation `Drink <: Liquid` after introducing `WriteOnlyList`?



Contravariance

Contravariance: Original subtyping direction is *reversed* in subtyping between “wrapped” versions (contra-)



`WriteOnlyList[T]` is *contravariant* on `T`

Subtyping Patterns

What happened to the original subtyping relation `Drink <: Liquid` after introducing `List`?

`Drink <: Liquid`



`List[Liquid] </: List[Drink]`
`List[Drink] </: List[Liquid]`

Invariance

Invariance: Original subtyping has *no implication* of subtyping between “wrapped” versions (in-)

`Drink <: Liquid`



`List[Liquid] </: List[Drink]`
`List[Drink] </: List[Liquid]`

`List[T]` is *invariant* on `T` (neither covariant nor contravariant)

Bivariance

Bivariance: Both *covariance* and *contravariance* apply in subtyping between “wrapped” versions (bi-)

`Drink <: Liquid`

↓

`List[Liquid] <: List[Drink]`
`List[Drink] <: List[Liquid]`

Most permissive, but allows type errors for certain cases

Type Variance

Type variance: How subtyping of types relates to their more “complex” versions

Given types $S <: T$:

Variance	Relation between <code>Wrapper[S]</code> and <code>Wrapper[T]</code>
Covariant	<code>Wrapper[S] <: Wrapper[T]</code> (carried over)
Contravariant	<code>Wrapper[T] <: Wrapper[S]</code> (carried over in “reverse”)
Invariant	Neither covariant nor contravariant (does not carry over)
Bivariant	Both covariant and contravariant (goes both ways)

Type Variance in Python

Type	Mutability	Variance
<code>list [...]</code> , <code>set [...]</code> , <code>dict [...]</code> , <code>Collection [...]</code> , <code>MutableSequence [...]</code> , <code>MutableSet [...]</code> , <code>MutableMapping [...]</code>	Mutable	Invariant (inflexible)
<code>Sequence [...]</code> , <code>Set [...]</code> , <code>Mapping [...]</code>	Immutable	Covariant (more flexible)

Prefer annotating *parameters* using *immutable* versions when applicable

Self-check

Recall that `object` is a *supertype* of all Python types

```
def safe_head(elems: list[object]) -> object | None:
    if elems:
        return elems[0]

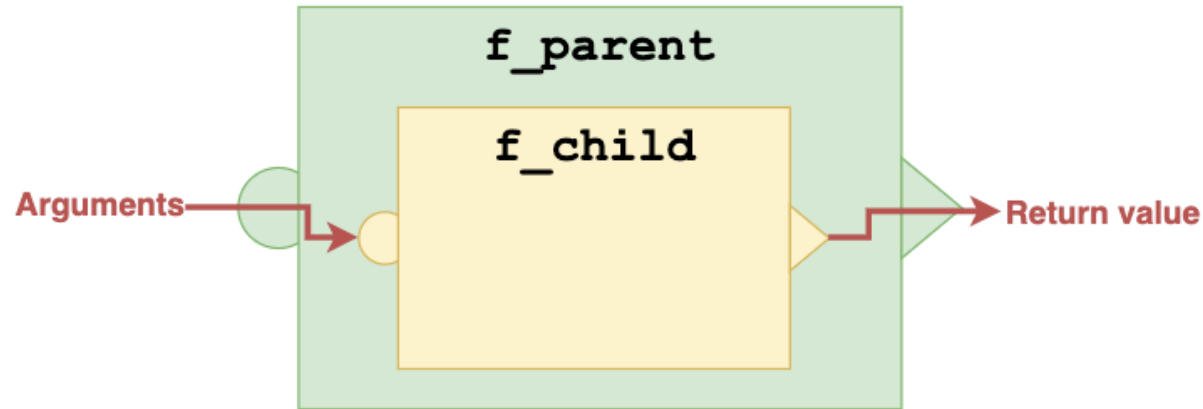
strs = ['a', 'b', 'c']
head = safe_head(strs)

if head:
    print(head.upper())
```

What is wrong with this?

Function Subtyping

Given two functions $f_child(\dots) \rightarrow \dots$ and $f_parent(\dots) \rightarrow \dots$, $f_child <: f_parent$ can be thought of as allowing this:



Happens for *method overriding* in inheritance

Subtyping of Function Parameters

Given: `Drink <: Liquid`

- Recall: If `S <: T`, then `S` can be used in place of `T`
 - **Case 1:** Should `f(x: Drink) <: f(x: Liquid)`?
 - **Case 2:** Should `f(x: Liquid) <: f(x: Drink)`?

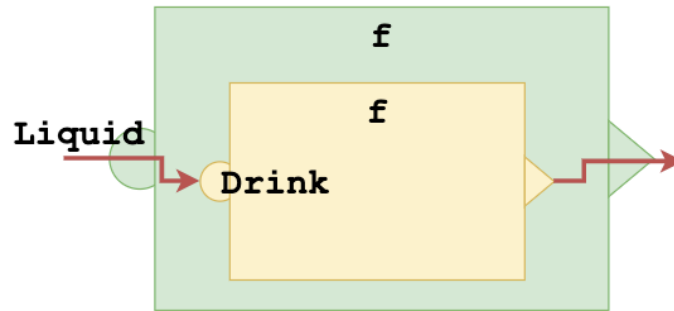
Which of the above will not break anything when allowed?

Note that the functions do not return anything (i.e., `None` is returned)

Subtyping of Function Parameters

Case 1: Should $f(x: \text{Drink}) <: f(x: \text{Liquid})$?

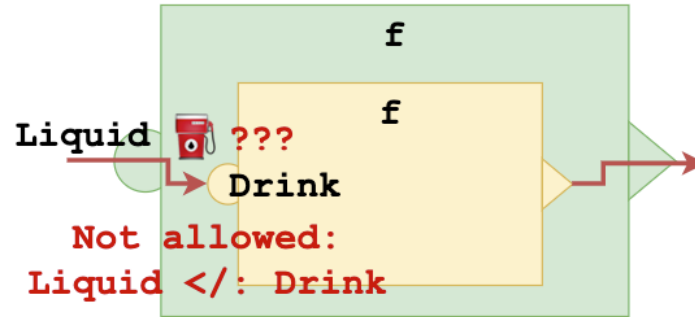
What kind of type variance is this given that $\text{Drink} <: \text{Liquid}$?



Is it possible to break this setup (i.e., make the inner f trigger a type error)?

Subtyping of Function Parameters

Case 1: Should $f(x: \text{Drink}) <: f(x: \text{Liquid})$?



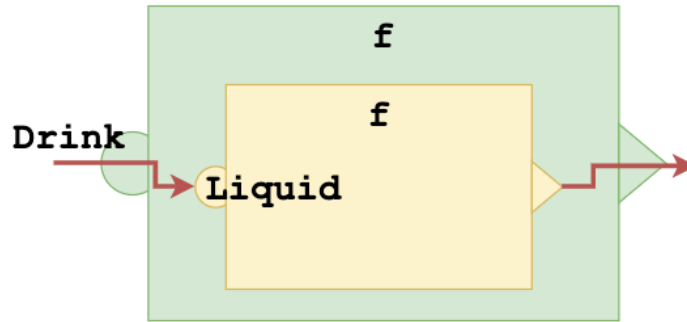
Is it possible to break this setup (i.e., make the inner f trigger a type error)?

Yes; function parameters should *not* be covariant!

Subtyping of Function Parameters

Case 2: Should $f(x: \text{Liquid}) <: f(x: \text{Drink})$?

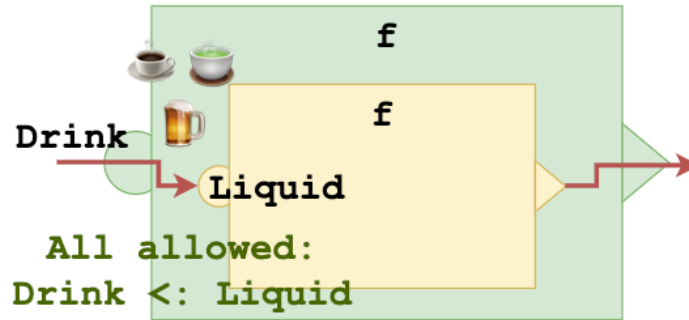
What kind of type variance is this given that $\text{Drink} <: \text{Liquid}$?



Is it possible to break this setup (i.e., make the inner f trigger a type error)?

Subtyping of Function Parameters

Case 2: Should `f(x: Liquid) <: f(x: Drink)`?



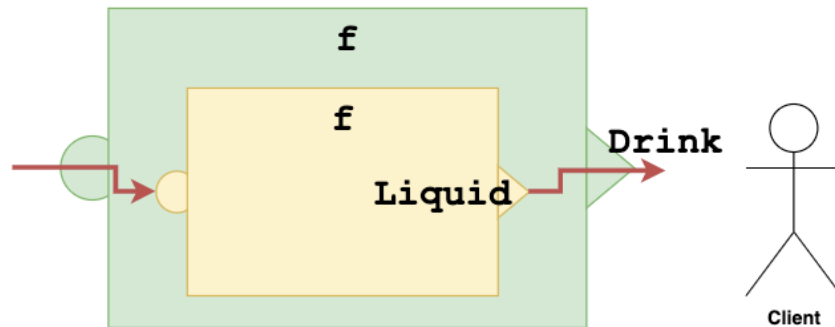
Is it possible to break this setup (i.e., make the inner `f` trigger a type error)?

This *always* works; function parameters should be *contravariant*

Subtyping of Return Types

Case 1: Should $f() \rightarrow \text{Liquid} <: f() \rightarrow \text{Drink}$?

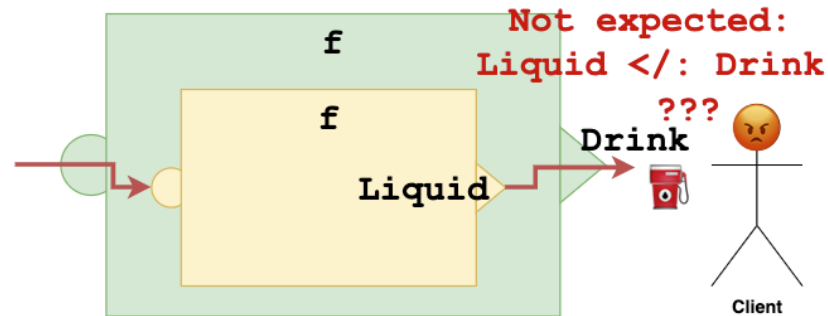
What kind of type variance is this given that $\text{Drink} <: \text{Liquid}$?



Is it possible to break this setup (i.e., make the client mad by betraying their expectations)?

Subtyping of Return Types

Case 1: Should $f() \rightarrow \text{Liquid} <: f() \rightarrow \text{Drink}$?



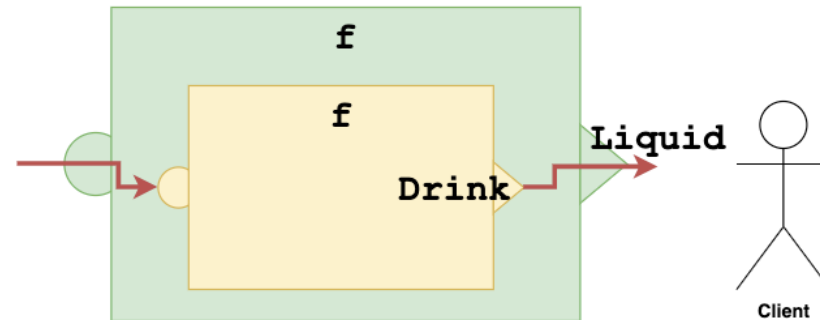
Is it possible to break this setup (i.e., make the client mad by betraying their expectations)?

Yes; return types should *not* be contravariant!

Subtyping of Return Types

Case 2: Should $f() \rightarrow \text{Drink} <: f() \rightarrow \text{Liquid}$?

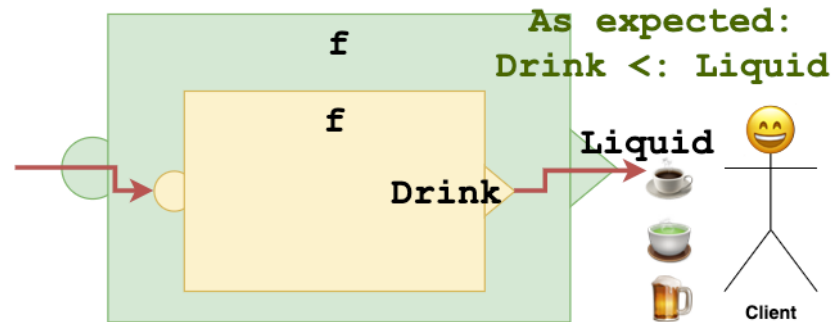
What kind of type variance is this given that $\text{Drink} <: \text{Liquid}$?



Is it possible to break this setup (i.e., make the client mad by betraying their expectations)?

Subtyping of Return Types

Case 2: Should $f() \rightarrow \text{Drink} <: f() \rightarrow \text{Liquid}$?



Is it possible to break this setup (i.e., make the client mad by betraying their expectations)?

This *always* works; return types should be *covariant*!

Subtyping of Functions

Function component	Type variance	Guideline
Parameters	Contravariant	“Require at most what is given”
Return types	Covariant	“Give at least what is promised”